

Problem Session 4

Memory Class and API in C Programming

Kiatisak Markmeeshap 67070501005
Tithinan Sobking 67070501018
Worawut Sereethai 67070501040
Chanya Poolketkij 67070501058

Contents

1	Environment and Compilation	2
2	Static Storage Class	2
2.1	Original program	2
2.2	Results from three runs	2
2.3	Program after removing <code>static</code>	3
2.4	Comparison with <code>-no-pie</code>	3
3	Extern Storage Class	4
3.1	Original program	4
3.2	Removing <code>extern</code> in <code>main()</code>	4
4	Memory Allocation	5
4.1	Original program	5
4.2	Meaning of the addresses	6
4.3	After uncommenting the allocation of <code>b</code>	7
5	Discussion	8
6	Conclusion	8
A	Complete Experiment Outputs	8

1 Environment and Compilation

All experiments were performed in Ubuntu running under WSL. The source files, executables, and output logs were kept in the same working directory.

```
cd ~/cpe333/ps4
gcc -Wall -Wextra -std=c11 -O0 -g -o static_lab static_lab.c
gcc -Wall -Wextra -std=c11 -O0 -g -o static_lab_nopie static_lab.c -no-pie
./static_lab
./static_lab_nopie
```

Listing 1: Working directory and compilation examples

The programs use standard C entry points with `int main(void)`. Pointer values are cast to `void *` when printed with the `%p` format specifier.

2 Static Storage Class

2.1 Original program

The original program declares `y` as a static local variable. It is initialized once and keeps its value between loop iterations.

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     auto int x = 3;
6
7     while (x > 0)
8     {
9         static int y = 5;
10        y++;
11
12        printf("The value of y is %d\n", y);
13        printf("The address of y is %p\n\n", (void *)&y);
14
15        x--;
16    }
17
18    return 0;
19 }
```

Listing 2: Static storage class program

2.2 Results from three runs

The program was compiled and executed three times without `-no-pie`. The address of `y` stays constant within one execution, while its address can change between executions because of position-independent loading and address randomization.

```
The value of y is 6
The address of y is 0x5eb92dd39010

The value of y is 7
The address of y is 0x5eb92dd39010
```

```
The value of y is 8
The address of y is 0x5eb92dd39010
```

Listing 3: Selected output from the original static program

The complete three-run output is included in Appendix A.

2.3 Program after removing static

After removing the keyword, `y` becomes an automatic local variable. It is initialized to 5 again on every loop iteration, so the printed value is 6 on every iteration.

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     auto int x = 3;
6
7     while (x > 0)
8     {
9         int y = 5;
10        y++;
11
12        printf("The value of y is %d\n", y);
13        printf("The address of y is %p\n\n", (void *)&y);
14
15        x--;
16    }
17
18    return 0;
19 }
```

Listing 4: Static keyword removed

```
The value of y is 6
The address of y is 0x7ffde7b84c40

The value of y is 6
The address of y is 0x7ffde7b84c40

The value of y is 6
The address of y is 0x7ffde7b84c40
```

Listing 5: Output after removing static

The address remains the same during this execution because the same stack location is reused by the loop. It is not guaranteed to be the same across separate executions.

2.4 Comparison with `-no-pie`

With the static program compiled using `-no-pie`, the static variable was located at the same address in all three runs in this environment: `0x404018`. The values still followed the same sequence 6, 7, and 8. For the non-static program, the stack address still changed between runs, because disabling PIE does not disable stack address randomization.

3 Extern Storage Class

3.1 Original program

The global variable `x` is defined once and referenced from both `main()` and `display()` using `extern`. Both functions therefore access the same object.

```
1 #include <stdio.h>
2
3 int x = 20;
4
5 static void display(void)
6 {
7     extern int x;
8     printf("Display value of x: %d\n", x);
9     printf("Address of x (in display function): %p\n\n", (void *)&x);
10 }
11
12 int main(void)
13 {
14     extern int x;
15     printf("Print value of x: %d\n", x);
16     printf("Address of x (in main function): %p\n\n", (void *)&x);
17     display();
18     return 0;
19 }
```

Listing 6: Extern storage class program

```
Print value of x: 20
Address of x (in main function): 0x5bb79ed55010

Display value of x: 20
Address of x (in display function): 0x5bb79ed55010
```

Listing 7: Original extern output

The value and address are identical in both functions. With the default PIE build, the global address may change between executions. With `-no-pie`, the global address was stable at `0x404018` in this environment.

3.2 Removing `extern` in `main()`

The modified line becomes `int x;`. This declares a new local variable that hides the global variable. It is not initialized, so reading it produces an indeterminate value and the compiler warns about using an uninitialized variable. The `display()` function still accesses the global value 20.

```
1 #include <stdio.h>
2
3 int x = 20;
4
5 static void display(void)
6 {
7     extern int x;
8     printf("Display value of x: %d\n", x);
9     printf("Address of x (in display function): %p\n\n", (void *)&x);
```

```

10 }
11
12 int main(void)
13 {
14     int x;
15     printf("Print value of x: %d\n", x);
16     printf("Address of x (in main function): %p\n\n", (void *)&x);
17     display();
18     return 0;
19 }

```

Listing 8: Extern removed from main

```

Print value of x: 32767
Address of x (in main function): 0x7fff7d990da4

Display value of x: 20
Address of x (in display function): 0x5ca88dea4010

```

Listing 9: Output after removing extern in main

The exact value printed by `main()` is not predictable and may differ between runs. The important observation is that the two functions now refer to different objects.

4 Memory Allocation

4.1 Original program

The program creates an integer pointer `a`, a floating-point pointer `b`, and a fixed-size array `c`. It then allocates memory for `a` using `malloc()` and expands it using `realloc()`.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(void)
5 {
6     int *a;
7     float *b;
8     int c[10];
9
10    printf("\nAddress of Pointer\n");
11    printf(">>%p\n", (void *)&a);
12    printf(">>%p\n", (void *)&b);
13
14    printf("\nEffective Address\n");
15    printf(">>%p\n", (void *)a);
16    printf(">>%p\n", (void *)b);
17
18    a = malloc(10 * sizeof(int));
19    printf("\nAfter malloc Pointer a\n");
20    printf(">>%p\n", (void *)a);
21    printf(">>%p\n", (void *)&a[0]);
22    printf(">>%p\n", (void *)&a[9]);
23
24    printf("\nArray c\n");
25    printf(">>%p\n", (void *)c);
26    printf(">>%p\n", (void *)&c[0]);

```

```

27     printf(">>%p\n", (void *)&c[9]);
28
29     a = realloc(a, 1000 * sizeof(int));
30     printf("\nAfter realloc Pointer a\n");
31     printf(">>%p\n", (void *)a);
32     printf(">>%p\n", (void *)&a[0]);
33     printf(">>%p\n", (void *)&a[9]);
34     printf(">>%p\n", (void *)&a[999]);
35
36     free(a);
37     return 0;
38 }

```

Listing 10: Memory allocation program

4.2 Meaning of the addresses

- `&a` and `&b` are the addresses of the pointer variables themselves.
- `a` and `b` are the addresses stored inside the pointers.
- After `malloc()`, `a` points to a block on the heap.
- `c` and `&c[0]` have the same base address because an array name decays to a pointer to its first element.
- `&a[9]` and `&c[9]` are offset by nine integer elements from the first element.
- `realloc()` may keep the original block or move it to a new location if a larger contiguous block is required.

```

Address of Pointer
>>0x7ffd5e8f0be0
>>0x7ffd5e8f0be8

Effective Address
>>(nil)
>>(nil)

After malloc Pointer a
>>0x6467b2cec2b0
>>0x6467b2cec2b0
>>0x6467b2cec2d4

Array c
>>0x7ffd5e8f0bf0
>>0x7ffd5e8f0bf0
>>0x7ffd5e8f0c14

After realloc Pointer a
>>0x6467b2cec2b0
>>0x6467b2cec2b0
>>0x6467b2cec2d4
>>0x6467b2ced24c

```

Listing 11: Memory allocation output before enabling b

The initial effective addresses are not valid allocated objects because the pointers have not been initialized. Their printed values are therefore not portable or predictable. The observed (nil) output is only one result from this execution.

4.3 After uncommenting the allocation of b

The second version allocates ten floating-point elements for `b` before calling `realloc()` on `a`.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(void)
5 {
6     int *a;
7     float *b;
8     int c[10];
9
10    printf("\nAddress of Pointer\n");
11    printf(">>%p\n", (void *)&a);
12    printf(">>%p\n", (void *)&b);
13
14    printf("\nEffective Address\n");
15    printf(">>%p\n", (void *)a);
16    printf(">>%p\n", (void *)b);
17
18    a = malloc(10 * sizeof(int));
19    printf("\nAfter malloc Pointer a\n");
20    printf(">>%p\n", (void *)a);
21    printf(">>%p\n", (void *)&a[0]);
22    printf(">>%p\n", (void *)&a[9]);
23
24    printf("\nArray c\n");
25    printf(">>%p\n", (void *)c);
26    printf(">>%p\n", (void *)&c[0]);
27    printf(">>%p\n", (void *)&c[9]);
28
29    b = malloc(10 * sizeof(float));
30    printf("\nAfter malloc Pointer b\n");
31    printf(">>%p\n", (void *)b);
32    printf(">>%p\n", (void *)&b[0]);
33    printf(">>%p\n", (void *)&b[9]);
34
35    a = realloc(a, 1000 * sizeof(int));
36    printf("\nAfter realloc Pointer a\n");
37    printf(">>%p\n", (void *)a);
38    printf(">>%p\n", (void *)&a[0]);
39    printf(">>%p\n", (void *)&a[9]);
40    printf(">>%p\n", (void *)&a[999]);
41
42    free(a);
43    free(b);
44    return 0;
45 }
```

Listing 12: Memory allocation with `b` enabled

```

After malloc Pointer b
>>0x57232be552e0
>>0x57232be552e0
>>0x57232be55304

After realloc Pointer a
>>0x57232be55310
>>0x57232be55310
>>0x57232be55334
>>0x57232be562ac

```

Listing 13: Memory allocation output after enabling b

In this run, allocating **b** occupied space immediately after the first allocation for **a**. When **a** was expanded, the allocator moved it to another address. This behavior is implementation-dependent, so another run or another machine may produce different addresses.

5 Discussion

The experiments show that storage class affects lifetime, linkage, and the location used by a variable. A static local variable retains its value after a function block is left, while an automatic variable is initialized again on each entry to the block. The **extern** declaration allows code in different functions or source files to refer to one global object.

The address experiments also show the effect of process memory layout. PIE allows the executable's data to be relocated between runs, while **-no-pie** makes the executable's fixed data address stable in this environment. Heap addresses remain controlled by the memory allocator and can change when **malloc()** or **realloc()** changes the allocation layout.

6 Conclusion

Static storage preserves a variable's value for the lifetime of the program. Extern storage provides access to a global object from another scope. Finally, **malloc()** and **realloc()** demonstrate dynamic heap allocation, while the address of an array declared inside a function belongs to the stack.

A Complete Experiment Outputs

```

STATIC original, default PIE - run 1
The value of y is 6
The address of y is 0x5887139cf010

The value of y is 7
The address of y is 0x5887139cf010

The value of y is 8
The address of y is 0x5887139cf010

STATIC original, default PIE - run 2

```

```
The value of y is 6
The address of y is 0x5f909d714010

The value of y is 7
The address of y is 0x5f909d714010

The value of y is 8
The address of y is 0x5f909d714010

STATIC original, default PIE - run 3
The value of y is 6
The address of y is 0x562ec98d1010

The value of y is 7
The address of y is 0x562ec98d1010

The value of y is 8
The address of y is 0x562ec98d1010

STATIC no static, default PIE - run 1
The value of y is 6
The address of y is 0x7ffdc82832f0

The value of y is 6
The address of y is 0x7ffdc82832f0

The value of y is 6
The address of y is 0x7ffdc82832f0

STATIC no static, default PIE - run 2
The value of y is 6
The address of y is 0x7ffd39556c20

The value of y is 6
The address of y is 0x7ffd39556c20

The value of y is 6
The address of y is 0x7ffd39556c20

STATIC no static, default PIE - run 3
The value of y is 6
The address of y is 0x7ffdb46998b0

The value of y is 6
The address of y is 0x7ffdb46998b0

The value of y is 6
The address of y is 0x7ffdb46998b0

STATIC original, -no-pie - run 1
The value of y is 6
The address of y is 0x404018

The value of y is 7
The address of y is 0x404018

The value of y is 8
The address of y is 0x404018
```

```

STATIC original, -no-pie - run 2
The value of y is 6
The address of y is 0x404018

The value of y is 7
The address of y is 0x404018

The value of y is 8
The address of y is 0x404018

STATIC original, -no-pie - run 3
The value of y is 6
The address of y is 0x404018

The value of y is 7
The address of y is 0x404018

The value of y is 8
The address of y is 0x404018

STATIC no static, -no-pie - run 1
The value of y is 6
The address of y is 0x7ffeea4a1110

The value of y is 6
The address of y is 0x7ffeea4a1110

The value of y is 6
The address of y is 0x7ffeea4a1110

STATIC no static, -no-pie - run 2
The value of y is 6
The address of y is 0x7ffd83e15e60

The value of y is 6
The address of y is 0x7ffd83e15e60

The value of y is 6
The address of y is 0x7ffd83e15e60

STATIC no static, -no-pie - run 3
The value of y is 6
The address of y is 0x7fff9b007c30

The value of y is 6
The address of y is 0x7fff9b007c30

The value of y is 6
The address of y is 0x7fff9b007c30

```

Listing 14: Complete static experiment output

```

EXTERN original, default PIE - run 1
Print value of x: 20
Address of x (in main function): 0x58e748aa6010

Display value of x: 20

```

```

Address of x (in display function): 0x58e748aa6010

EXTERN original, default PIE - run 2
Print value of x: 20
Address of x (in main function): 0x6216efdc6010

Display value of x: 20
Address of x (in display function): 0x6216efdc6010

EXTERN original, default PIE - run 3
Print value of x: 20
Address of x (in main function): 0x6171c3ddc010

Display value of x: 20
Address of x (in display function): 0x6171c3ddc010

EXTERN original, -no-pie - run 1
Print value of x: 20
Address of x (in main function): 0x404018

Display value of x: 20
Address of x (in display function): 0x404018

EXTERN original, -no-pie - run 2
Print value of x: 20
Address of x (in main function): 0x404018

Display value of x: 20
Address of x (in display function): 0x404018

EXTERN original, -no-pie - run 3
Print value of x: 20
Address of x (in main function): 0x404018

Display value of x: 20
Address of x (in display function): 0x404018

EXTERN removed in main, default PIE - run 1
Print value of x: 32765
Address of x (in main function): 0x7ffd7570a044

Display value of x: 20
Address of x (in display function): 0x6401920cd010

EXTERN removed in main, default PIE - run 2
Print value of x: 32765
Address of x (in main function): 0x7ffd260ef104

Display value of x: 20
Address of x (in display function): 0x5802dce82010

EXTERN removed in main, default PIE - run 3
Print value of x: 32764
Address of x (in main function): 0x7ffc81b49474

Display value of x: 20
Address of x (in display function): 0x57b023182010

```

```

EXTERN removed in main, -no-pie - run 1
Print value of x: 32764
Address of x (in main function): 0x7ffc4a52d794

Display value of x: 20
Address of x (in display function): 0x404020

EXTERN removed in main, -no-pie - run 2
Print value of x: 32766
Address of x (in main function): 0x7ffe2204d474

Display value of x: 20
Address of x (in display function): 0x404020

EXTERN removed in main, -no-pie - run 3
Print value of x: 32766
Address of x (in main function): 0x7ffe612fae34

Display value of x: 20
Address of x (in display function): 0x404020

```

Listing 15: Complete extern experiment output

```

MEMORY original, no -no-pie

Address of Pointer
>>0x7ffdcae952b0
>>0x7ffdcae952b8

Effective Address
>>(nil)
>>(nil)

After malloc Pointer a
>>0x5b5db992c2b0
>>0x5b5db992c2b0
>>0x5b5db992c2d4

Array c
>>0x7ffdcae952c0
>>0x7ffdcae952c0
>>0x7ffdcae952e4

After realloc Pointer a
>>0x5b5db992c2b0
>>0x5b5db992c2b0
>>0x5b5db992c2d4
>>0x5b5db992d24c
MEMORY with b uncommented, no -no-pie

Address of Pointer
>>0x7ffc24be13d0
>>0x7ffc24be13d8

Effective Address
>>(nil)
>>(nil)

```

```
After malloc Pointer a
>>0x5a42ae8592b0
>>0x5a42ae8592b0
>>0x5a42ae8592d4

Array c
>>0x7ffc24be13e0
>>0x7ffc24be13e0
>>0x7ffc24be1404

After malloc Pointer b
>>0x5a42ae8592e0
>>0x5a42ae8592e0
>>0x5a42ae859304

After realloc Pointer a
>>0x5a42ae859310
>>0x5a42ae859310
>>0x5a42ae859334
>>0x5a42ae85a2ac
```

Listing 16: Complete memory experiment output